



Disciplina Técnicas de Programação 1

Profa. Roberta Barbosa Oliveira

Sistema de Gestão de Copa do Mundo

Grupo 9 – 2026.1

Danilo Lins Castro – 251019735 – 251019735@aluno.unb.br
Helton Rodrigues da Silva Júnior – 251021789 – 251021789@aluno.unb.br
Ighor Gomes das Chagas – 221020913 – 221020913@aluno.unb.br
João Pedro Barros Silva Azevedo – 251004311 – 251004311@aluno.unb.br
João Vítor da Silva Lima – 251023101 – 251023101@aluno.unb.br

Resumo

Nesta entrega, o grupo focou no entedimento da proposta e de seus requisitos funcionais e não-funcionais. Ademais, decidiram-se aspectos como a divisão de tarefas, o ambiente de desenvolvimento e as ferramentas de trabalho em equipe. Por fim, produziram-se as principais telas do sistema, com foco apenas em seu design e integração.

1 Introdução

1.1 Contexto do Projeto

Neste projeto, busca-se desenvolver um sistema de gestão da Copa do Mundo de futebol. Assim, espera-se tratar de aspectos naturais do universo desse esporte, como partidas, seleções, jogadores, árbitros e público, todos mutuamente dependentes e essenciais para o bom funcionamento da plataforma. Para isso, o grupo lançará mão de conhecimentos obtidos ao longo da disciplina de Técnicas de Programação 1, do Departamento de Ciência da Computação, de modo a abordar, principalmente, tópicos relacionados a Programação Orientada a Objetos.

1.2 Objetivos de Desenvolvimento

De maneira geral, o grupo espera desenvolver um sistema coerente, coeso e que cumpre o problema proposto com excelência. Assim, valorizaram-se, especialmente, a modularização, a

qualidade de software e a robustez do projeto, fatores alcançados por meio da Programação Orientada a Objetos e tudo que a circunda, como polimorfismo, encapsulamento, heranças, tratamento de exceções e padrões de desenvolvimento. Ademais, os alunos almejam, além do desenvolvimento individual, aperfeiçoar-se no trabalho em equipe, com foco na produção conjunta e no uso de ferramentas colaborativas dominantes no mercado.

1.3 Requisitos Funcionais

1.3.1 Administração/Gestão

- Cadastrar, editar, excluir e listar contas de usuários do sistema (administradores, organizadores, árbitros, operadores)
- Restringir funcionalidades de acordo com o perfil de acesso.
- Permitir pesquisa de usuários por múltiplos critérios (nome, função, país, status).
- Validar dados de usuários (e-mail, identificação).
- Gerar relatórios consolidados da competição (número de partidas, público, desempenho de seleções).

1.3.2 Seleções e Jogadores

- Cadastrar, editar, excluir e listar seleções (país, grupo, técnico).
- Cadastrar, editar, excluir e listar jogadores (nome, posição, número, idade, seleção).
- Associar jogadores a uma seleção.
- Controlar status dos jogadores (ativo, lesionado, suspenso).
- Consultar seleções e jogadores por critérios (grupo, posição, status).

1.3.3 Estádios e Arbitragem

- Cadastrar, editar, excluir e listar estádios (nome, localização, capacidade).
- Associar partidas a estádios.
- Cadastrar árbitros (nome, nacionalidade, experiência).
- Designar árbitros para partidas.
- Consultar estádios e árbitros por critérios.

1.3.4 Partidas

- Cadastrar, editar, excluir e listar partidas (data, horário, estádio, seleções).
- Definir fase da competição (grupos, oitavas, quartas, semifinal, final).
- Registrar resultados das partidas (placar, eventos).
- Atualizar status das partidas (agendada, em andamento, finalizada).
- Consultar partidas por seleção, fase ou data.

1.3.5 Ingressos e Público

- Gerenciar venda de ingressos por partida.
- Definir categorias de ingressos (VIP, padrão, meia-entrada).
- Controlar quantidade disponível por partida.
- Registrar compra de ingressos.
- Gerar relatórios de público e arrecadação.

1.4 Requisitos não Funcionais

- Ter interface amigável para interação (por exemplo: Swing, JavaFX ou menu de opções em prompt).
- Oferecer usabilidade adequada (botões e menus claros, feedback ao usuário, alertas antes de exclusão definitiva).
- Exibir mensagens claras para erros de regra de negócio, usando exceções personalizadas.
- Deve ser implementado de forma orientada a objetos, garantindo modularidade e permitindo a extensão de funcionalidades sem impactar de forma significativa as classes já existentes.
- A modelagem deve priorizar herança e composição para reaproveitamento de código e redução de duplicidade.
- Deve respeitar princípios de encapsulamento, expondo apenas os métodos necessários ao uso externo.
- Deve adotar boas práticas de POO, como responsabilidade única (altamente coesa) e baixo acoplamento entre módulo, para facilitar manutenção e evolução do sistema.
- Empregar pelo menos dois padrões de projeto (por exemplo: Singleton, Factory, Builder).
- Ser implementado em camadas bem definidas (por exemplo: apresentação, aplicação, domínio, persistência, infraestrutura).
- Armazenar dados em arquivos (por exemplo: texto, binário ou serialização de objetos).
- Ser executável em qualquer sistema operacional com Java instalado.
- Garantir autenticação obrigatória via login e senha.
- (Opcional) Suporte a concorrência, por exemplo para venda de ingressos.

1.5 Regras de negócio

1.5.1 Administração/Gestão

- Perfis de acesso são hierárquicos: administrador (acesso total), organizador (gestão de partidas e seleções), árbitro (visualização de partidas designadas), operador (ingressos e registros).
- Administradores podem acessar todas as categorias, enquanto organizador, árbitro e operador têm acesso restrito conforme perfil.
- Apenas administradores podem criar, editar ou remover contas de outros usuários.
- Senhas devem respeitar políticas mínimas (por exemplo: 8 caracteres, letras e números).

1.5.2 Seleções e Jogadores

- Cada jogador deve estar vinculado a uma única seleção.
- Uma seleção deve possuir número mínimo e máximo de jogadores (exemplo: 18 a 26).
- Jogadores suspensos ou lesionados não podem participar de partidas.

1.5.3 Estádio e Arbitragem

- Um estádio não pode sediar duas partidas no mesmo horário.
- Cada partida deve ter ao menos um árbitro principal.
- Árbitros não podem atuar em partidas de seleções de sua nacionalidade.

1.5.4 Partidas

- Uma partida deve envolver exatamente duas seleções diferentes.
- Uma seleção não pode jogar duas partidas no mesmo horário.
- Partidas devem ser associadas a um estádio.
- Apenas partidas finalizadas podem ter resultados registrados definitivamente.

1.5.5 Ingressos e Público

- A quantidade de ingressos não pode exceder a capacidade do estádio.
- Ingressos não podem ser vendidos após o início da partida.
- Cada ingresso deve estar vinculado a uma partida específica.
- Relatórios financeiros devem considerar apenas ingressos vendidos.

1.5.6 Atores

- **Administrador:** Responsável pelo controle geral do sistema, usuários e configurações.
- **Organizador:** Gerencia seleções, jogadores, partidas e estádios.
- **Árbitro:** Visualiza partidas designadas e informações relacionadas.
- **Operador:** Responsável pela venda de ingressos e controle de público.
- **Espectador:** Não acessa diretamente o sistema (representado no controle de ingressos).

1.6 Estrutura do Relatório

As próximas seções do relatório detalham a análise e o desenvolvimento do software implementado. Na Seção 2 são apresentadas a modelagem, a estrutura, os módulos, a arquitetura e as funcionalidades desenvolvidas, descrevendo como cada etapa do projeto contribuiu para a construção do software. Na Seção 3, discute-se a análise das funcionalidades desenvolvidas, destacando resultados alcançados, integração entre os módulos, confiabilidade, usabilidade e outras características do software, além de eventuais limitações ou desafios encontrados durante o desenvolvimento. Por fim, a Seção 4 sintetiza a relevância do trabalho desenvolvido, avaliando seu impacto no contexto aplicado e apontando possíveis melhorias e extensões para evoluções futuras.

Nos relatórios parciais, devem ser incluídas apenas as partes que já foram desenvolvidas até o momento. Cada entrega deve corresponder a uma etapa específica da seção de Solução Proposta: Estrutura Inicial, Design e Modelagem, Implementação da Lógica do Software e Integração e Navegabilidade. Nessas versões, é obrigatório incluir o Resumo e a Introdução, enquanto a Solução Proposta deve refletir a etapa de desenvolvimento atual, detalhando funcionalidades, diagramas e técnicas de programação aplicadas. Alterações ou melhorias em relação a versões anteriores devem ser documentadas na versão atual.

No relatório final, o documento deve ser revisado e atualizado, mantendo apenas o conteúdo referente ao que foi de fato desenvolvido, contemplando todas as implementações, funcionalidades, diagramas, módulos, técnicas de programação aplicadas e integração final. Além do Resumo, da Introdução e da Solução Proposta, o relatório final deve incluir as seções Resultados e Discussão, e Conclusão e Evolução Futura, para refletir a solução completa desenvolvida pelo grupo e sua relevância para o domínio aplicado.

2 Solução Proposta

2.1 Estrutura Inicial

Na definição das ferramentas, decidiu-se pelo Git e GitHub para controle de versionamento do projeto; NetBeans para Interface de Desenvolvimento Integrado (IDE), Maven para gestão e automação de dependências; e o Swing como biblioteca gráfica, em virtude da sua integração de design com a IDE. O repositório do projeto no GitHub está disponível em: <https://github.com/vitor2828/tp1>

A estruturação em pacotes foi definida conforme a seguinte disposição:

```
sistema.gestao.copa.do.mundo/  
  src/  
    administracao.gestao/  
      gui  
      classes  
      excecoes  
    selecao.jogador/  
      gui  
      classes  
      excecoes  
    estadio.arbitragem/  
      gui  
      classes  
      excecoes  
    partidas/  
      gui  
      classes  
      excecoes  
    ingressos.publico/  
      gui  
      classes  
      excecoes  
  README.md
```

O desenvolvimento foi repartido entre os membros constituintes segundo a distribuição abaixo.

- **João Vitor - Administração e Gestão:** gestão de usuários, autenticação, controle de acesso e relatórios gerais. Classes: Usuario, Administrador, Organizador, Operador. Telas: Login, Gestão de Usuários, Relatórios, Tela Inicial
- **João Pedro - Seleções e Jogadores:** gestão de seleções e elenco de jogadores. Classes: Selecao, Jogador. Telas: Cadastro de Seleções, Cadastro de Jogadores, Consulta
- **Helton Rodrigues - Estádios e Arbitragem:** gestão de estádios e árbitros. Classes: Estadio, Arbitro, DesignacaoArbitro. Telas: Cadastro de Estádios, Cadastro de Árbitros, Designação Árbitro
- **Danilo Lins - Partidas:** controle das partidas e resultados. Classes: Partida, Fase, Resultado. Telas: Cadastro de Partidas, Registro de Resultados, Consulta de Jogos.
- **Ighor Gomes - Ingressos e Público:** controle de ingressos e arrecadação. Classes: Ingresso, Venda, CategoriaIngresso. Telas: Venda de Ingressos, Controle de Público, Relatórios Financeiros

- **Classe UsuarioLogado:** Essa classe é utilizada para controlar a sessão atual da plataforma, central para validação de permissões. Com isso, contém apenas um atributo final do tipo Usuario, para que não seja possível modificar informações em outras partes do sistema.
- **Classe Papel:** Uma classe abstrata cujo objetivo principal é agrupar todos os cargos possíveis em uma estrutura só. Assim, não tem muito conteúdo, apenas um método abstrato que retorna o nome do cargo em questão.
- **Classe Permissao:** Da mesma maneira que Papel, é uma classe abstrata que agrupa as mais diversas permissões, as quais validam se um usuário pode efetuar certa ação.
- **Classe Organizador:** É uma especialização direta de Papel, caracterizando um usuário que pode criar/editar/excluir partidas, seleções, jogadores e árbitros. Com isso, seu conjunto de permissões faz jus a esses métodos.
- **Classe Operador:** É uma outra especialização direta de Papel. No entanto, suas permissões permitem o usuário realizar operações financeiras, como a venda de ingressos e a geração de relatórios.
- **Classe Administrador:** Essa classe tem acesso total ao sistema, ou seja, está no topo da hierarquia de usuários. Entretanto, possui a permissão exclusiva de criar/editar/excluir outros usuários. Dessa forma, também é uma especialização de Papel, mas com todas as permissões da plataforma.
- **Classe AdministraUsuario:** Sendo uma especialização de Permissao, AdministraUsuario contém todos os métodos relacionados ao CRUD de usuários. Se um Papel tem essa classe entre suas permissões, o Usuario ao qual pertence poderá realizar essas operações.
- **Classe GeraRelatorios:** Lógica análoga a AdministraUsuario, porém com métodos relacionadas à geração de relatórios administrativos, contendo o número de usuários cadastrados e seus tipos.

2.2.3 Módulo Jogadores e Seleções

- **Classe Jogador:** Essa classe é responsável por armazenar e editar os elementos inerentes a cada jogador. Destacam-se as enumerações Posicao e StatusJogador, usadas para padronizar a categorização de jogadores. Seu único método é um construtor parcial, o qual não recebe os elementos de performance (faltas, cartões vermelhos, amarelos, etc) como parâmetros.
- **Classe Selecao:** Classe encarregada de organizar os elementos de cada Seleção. Apresenta constantes de mínimo e máximo de membros constituintes, para atender às regras de negócio; e agrega o time em uma tabela Hash com tratamento de colisões por Lista Linkada para garantir escalabilidade do sistema de gestão como um todo. Vale ressaltar que o único construtor da classe modifica múltiplos jogadores, enquanto, os demais métodos alteram somente um jogador por chamada.

2.2.4 Estádios e Arbitragem

- **Classe Estadio:** Responsável por representar e armazenar as informações da infraestrutura, como nome, localização e capacidade. Além do método construtor para inicialização dos objetos, possui métodos de acesso e modificação (Getters e Setters) para garantir o encapsulamento dos atributos privados.
- **Classe Arbitro:** Representa os árbitros cadastrados no sistema, contendo atributos básicos de identificação e qualificação (como nacionalidade e nível de experiência). Assim como a classe Estádio, utiliza métodos Getters e Setters para controle seguro de acesso aos dados, além do seu construtor padrão.
- **Classe DesignacaoArbitro:** Atua como uma associação lógica. Seus únicos atributos são uma instância da classe **Arbitro** e um valor da enumeração **FuncaoArbitragem**, que são os elementos necessários para alocar a equipe de arbitragem para cada partida. O acesso aos seus dados também é protegido por encapsulamento, sendo inicializada através de seu método construtor.

2.2.5 Partidas

- **Classe Partida:** Essa classe tem o objetivo de armazenar os dados que representam cada partida dentro da competição como código, data, onde ocorreu, o árbitro designado e o andamento da partida. Os métodos para o cadastro das informações estão na classe **Organizador**, como dito anteriormente.
- **Classe Fase:** Responsável por representar as diferentes etapas da competição, como grupos, oitavas, quartas, semifinais e final. A classe utiliza uma enumeração para padronizar as fases disponíveis no sistema e possui um método de validação capaz de criar uma fase e verificar que ela está de acordo com outras etapas já criadas, garantindo consistência na organização da competição.
- **Classe Resultado:** Classe encarregada de armazenar e representar o desfecho de cada partida. Seus atributos contemplam informações como quantidade de gols de cada seleção, existência de desempate e seleção vencedora. Além disso, possui um método responsável por criar e validar os resultados registrados, assegurando coerência entre os dados estatísticos da partida e o resultado final apresentado.

2.2.6 Módulo Ingressos e Público

- **Classe Ingresso:** Essa classe representa os ingressos disponíveis para cada partida do sistema. Ela armazena informações importantes, como identificação, valor, setor do estádio e status da venda. Além disso, permite controlar os ingressos vendidos e disponíveis, ajudando na organização do acesso do público aos jogos.
- **Classe DadosIngresso:** Responsável por armazenar e organizar os dados relacionados aos ingressos e ao público das partidas. Entre as informações registradas, destacam-se quantidade de ingressos vendidos, renda arrecadada e ocupação do estádio. Dessa forma, a classe auxilia no controle estatístico e administrativo do sistema.

- **Classe OperacaoIngresso:** Essa classe é responsável pelas operações envolvendo os ingressos, como venda, cancelamento e validação. Seus métodos permitem organizar as transações realizadas no sistema, além de auxiliar no controle financeiro e operacional das partidas.

2.3 Implementação da Lógica do Software

2.3.1 Administração

Antes de tudo, é importante elucidar dois padrões de design que foram essenciais para arquitetar essa seção: o RBAC (Role-Based Access Control, ou Controle de Acesso Baseado em Papéis) e a Injeção de Dependência. Além disso, deve-se tecer alguns comentários acerca da escolha do modelo de persistência.

O primeiro é utilizado para restringir o acesso ao sistema a depender do tipo de usuário atual. Em geral, é definido por uma classe Papel, que, associada a um Usuario, é verificada para certificar a ação. Isso, na teoria, já é suficiente para estruturar uma lógica de permissões sólida. No entanto, para aumentar a modularidade e a escalabilidade da plataforma, optou-se por, em cada Papel, dispor de uma coleção de permissões, representadas pela classe Permissao. Isso permite que cada cargo possa ter suas permissões facilmente verificadas e modificadas sem alterar outras partes do código, de modo a seguir boas práticas de desenvolvimento de software, destacando-se o SOLID.

O segundo, por sua vez, é responsável por obter informações do usuarioLogado, de forma a alimentar o RBAC. Sob uma primeira análise, uma maneira simples de fazer isso seria transformar usuarioLogado em uma classe Singleton, isto é, instanciada apenas uma vez e com os atributos estáticos, o chamado Padrão Singleton. No entanto, essa lógica é vista por muitos como um anti-padrão de design, de forma a ser evitada em sistemas de larga escala por razões de segurança. Dessa maneira, considera-se a Injeção de Dependência o padrão ideal que substitui o Singleton: trata-se de, toda vez que uma parte do código exigir informações da sessão, passar UsuarioLogado como argumento da função. Assim, o objeto estará restrito ao escopo da função onde é chamado, preservando-o de possíveis problemas.

Por fim, quanto à persistência, o grupo escolheu o JSON (JavaScript Object Notation) com a biblioteca Jackson. Embora o JSON seja, obviamente, nativo do JavaScript, o uso da Jackson possibilitou a conversão fácil entre arquivos de texto e objetos. Isso, comparado com o uso de texto bruto, tornou o trabalho muito mais eficiente, visto que não houve necessidade de criar funções personalizadas para serializar e deserializar os objetos.

Com tudo isso dito, explica-se, agora, a implementação das principais funções referentes a esta seção. Primeiro, destacam-se as funções de AdministraUsuario, que, por padrão, são todas *static*.

```
static public boolean criaUsuario(Usuario usuarioCadastro) throws
    SenhaInsuficienteException {

    if (usuarioCadastro.getSenha().length() < 8) {
        throw new SenhaInsuficienteException("Senha nao cumpre requisitos
            minimos");
    }

    ObjectMapper mapper = new ObjectMapper();
```

```

File persistenciaUsuarios = new File(USUARIOS_FILE_PATH);

try {
    Map<String, Usuario> mapUsuarios = mapper.readValue(
        persistenciaUsuarios, new TypeReference<Map<String, Usuario>>() {});
    mapUsuarios.put(usuarioCadastro.getIdentificacao(), usuarioCadastro);
    mapper.writeValue(persistenciaUsuarios, mapUsuarios);
}

catch (JsonMappingException e) {
    System.err.println("Houve algum problema no mapeamento do JSON durante
        o cadastro do usuario.");
    return false;
}

catch (IOException e) {
    System.err.println("Houve algum problema ao manipular o arquivo
        durante o cadastro do usuario.");
    return false;
}

return true;
}

```

A função acima é a `criaUsuario()`. Ela recebe uma instancia de `Usuario` e retorna um booleano, que comunica o resultado da operação. Não há muito o que dizer: a função simplesmente inicia o arquivo como um *Map* de `Usuario`, adiciona o novo usuário com seu ID como *key* e reescreve na persistência. Em todas as funções dessa natureza, teremos as exceções `JsonMappingException`, para problemas na análise do JSON, e `IOException`, para problemas na manipulação do arquivo. Por fim, unicamente nesse método, teremos `SenhaInsuficienteException`, uma exceção que, como previsto nas regras de negócio, estipula requisitos mínimos para que a senha seja validada.

```

static public List<Usuario> listaUsuario() {
    ObjectMapper mapper = new ObjectMapper();
    File persistenciaUsuarios = new File(USUARIOS_FILE_PATH);
    List<Usuario> retornaListaUsuarios = new ArrayList<>();

    try {
        Map<String, Usuario> mapUsuarios = mapper.readValue(
            persistenciaUsuarios, new TypeReference<Map<String, Usuario>>() {});

        for (Map.Entry<String, Usuario> entry : mapUsuarios.entrySet()) {
            retornaListaUsuarios.add(entry.getValue());
        }
    }

    catch (JsonMappingException e) {
        System.err.println("Houve algum problema no mapeamento do JSON ao
            listar usuarios");
    }

    catch (IOException e) {
        System.err.println("Houve algum problema ao manipular o arquivo ao
            listar usuarios");
    }
}

```

```
        return retornaListaUsuarios;
    }
```

Essa função, por sua vez, é a `listaUsuarios()`. Ela inicia o arquivo JSON como *Map*, itera-o completamente e, a cada loop, adiciona o *Usuario* em questão a uma *List*, a qual será retornada. Essa solução foi escolhida pois é mais fácil integrar listas à interface gráfica do *Swing*.

```
static public List<Usuario> pesquisaUsuario(String nome, String identificacao,
    String email, String pais, String senha, Usuario.StatusUsuario status,
    Papel papel) {
    // TODO: provavelmente nao vai ser elegante criar um poliformismo por
    // overloading.
    // aqui, vou cuidar para, na tela, receber os argumentos. Se ele estiver
    // desabilitado, vou receber como null
    // para ignorar durante a busca.

    ObjectMapper mapper = new ObjectMapper();
    File persistenciaUsuarios = new File(USUARIOS_FILE_PATH);
    List<Usuario> retornaListaUsuarios = new ArrayList<>();

    try {
        Map<String, Usuario> mapUsuarios = mapper.readValue(
            persistenciaUsuarios, new TypeReference<Map<String, Usuario>>(){});

        // 1 caso: se a identificacao nao for nula, ha apenas um retorno.
        // Portanto, pode simplesmente ver se existe no JSON e retornar uma
        // lista unitaria
        // 2 caso: iterar e coletar os usuarios que coincidem com algum
        // parametro.

        if (identificacao.isEmpty() == false && mapUsuarios.containsKey(
            identificacao)) {
            retornaListaUsuarios.add(mapUsuarios.get(identificacao));
        }

        else {
            for (Map.Entry<String, Usuario> entry : mapUsuarios.entrySet()) {
                Usuario usuarioIteracao = entry.getValue();

                if (usuarioIteracao.getNome().equals(nome) || usuarioIteracao.
                    getEmail().equals(email) || usuarioIteracao.getPais().
                    equals(pais)
                    || usuarioIteracao.getStatus() == status ||
                    usuarioIteracao.getPapel().getClass() == papel.getClass()
                    ()) {
                    retornaListaUsuarios.add(usuarioIteracao);
                }

                // isso jah trata a duplicata, porque um usuario vai ser
                // analisado apenas uma vez. Melhor do que fazer um if ou
                // switch case para cada caso
            }
        }
    }
}
```

```

    }

    catch (JsonMappingException e) {
        System.err.println("Houve algum problema no mapeamento do JSON durante
                           a pesquisa de usuarios");
    }

    catch (IOException e) {
        System.err.println("Houve algum problema ao manipular o arquivo
                           durante a pesquisa de usuarios");
    }

    return retornaListaUsuarios;
}

```

Essa função é a `pesquisaUsuario()`. Da mesma maneira que a `listaUsuario()`, ela estrutura o *Map* da persistência e o percorre. No entanto, somente serão adicionados à *List* de retorno usuários que possuem algum dos parâmetros dados. Em geral, há alguns detalhes dignos de menção: a pesquisa por ID, por retornar apenas um usuário, foi tratada separadamente; não há necessidade de tratar duplicatas, visto que cada usuário do *Map* será iterado uma única vez; com a integração da interface gráfica, valores desconsiderados na busca serão passados como *null*, o que já trará a forma obtida.

```

static public boolean excluiUsuario(Usuario usuario) {
    ObjectMapper mapper = new ObjectMapper();
    File persistenciaUsuarios = new File(USUARIOS_FILE_PATH);

    try {
        Map<String, Usuario> mapUsuarios = mapper.readValue(
            persistenciaUsuarios, new TypeReference<Map<String, Usuario>>() {});
        mapUsuarios.remove(usuario.getIdentificacao());
        mapper.writeValue(persistenciaUsuarios, mapUsuarios);
    }

    catch (JsonMappingException e) {
        System.err.println("Houve algum problema no mapeamento do JSON ao
                           excluir usu rio");
        return false;
    }

    catch (IOException e) {
        System.err.println("Houve algum problema ao manipular o arquivo ao
                           excluir usu rio");
        return false;
    }

    return true;
}

```

Esse método é o `excluiUsuario()`. Trata-se de uma versão complementar do `criaUsuario`, visto que recebe um usuário como parâmetro, busca-o na persistência e o remove, de modo a, logo depois, atualizar o arquivo.

Agora, quanto à permissão `geraRelatorios`, temos um único método:

```

static public Map<String, Integer> contaUsuarios() {
    int[] vetorAuxiliar = {0, 0, 0, 0, 0};
}

```

```

ObjectMapper mapper = new ObjectMapper();
File persistenciaUsuarios = new File(USUARIOS_FILE_PATH);

try {
    Map<String, Usuario> mapUsuarios = mapper.readValue(
        persistenciaUsuarios, new TypeReference<Map<String, Usuario>>(){});

    for (Map.Entry<String, Usuario> entry : mapUsuarios.entrySet()) {
        vetorAuxiliar[0]++;

        switch (entry.getValue().apel) {
            case Administrador u -> {
                vetorAuxiliar[1]++;
            }

            case Organizador u -> {
                vetorAuxiliar[2]++;
            }

            case Operador u -> {
                vetorAuxiliar[3]++;
            }

            default -> {
                // nao faz nada
            }
        }
    }
}

catch (JsonMappingException e) {
    System.err.println("Houve algum problema no mapeamento do JSON ao
        gerar relat rios de usu rio");
}

catch (IOException e) {
    System.err.println("Houve algum problema ao manipular o arquivo ao
        gerar relat rios de usu rio");
}

Map<String, Integer> mapaUsuarios = Map.of (
    "numTotalUsuarios", vetorAuxiliar[0],
    "numAdministradores", vetorAuxiliar[1],
    "numOrganizadores", vetorAuxiliar[2],
    "numOperadores", vetorAuxiliar[3],
    "numArbitros", vetorAuxiliar[4]
);

return mapaUsuarios;
}

```

Esse é o `contaUsuarios()`, cuja função é simplesmente iterar a persistência de usuários e contar cada tipo. Retorna um outro *Map*, dessa vez de `String` e `Integer`, respectivamente a chave e o valor, para que os resultados sejam mais fáceis de interpretar e, conseqüentemente, integrar com as telas.

2.3.2 Jogadores e Seleções

Foram exigidas três regras de negócio para esse tópico:

1. Cada jogador deve estar vinculado a uma única seleção;
2. Uma seleção deve possuir número mínimo e máximo de jogadores (exemplo: 18 a 26);
3. Jogadores suspensos ou lesionados não podem participar de partidas.

As regras 1. e 2. são garantidas pela responsabilização exclusiva da classe `Selecao` sobre a modificação do atributo `selecao` de cada jogador. Mais especificamente através da adaptação do método `setTime()` e dos métodos personalizados `convocarJogador()` e `dispensarJogador()`.

Para impedir a instanciação de seleções que fujam das regras 1. e 2., o método `setTime()` foi alterado para somente modificar o atributo `Selecao.time` se as condições abaixo forem atendidas. Caso contrário, é lançada uma exceção de `IllegalArgumentException`.

1. Possuir quantidade de elementos entre o mínimo e máximo estabelecidos. No caso, entre 18 e 26 inclusive;
2. Nenhum jogador estiver previamente vinculado a outra seleção. Isto é, todos os jogadores possuem `selecao = null`.

Ademais, `convocarJogador()` acrescenta o `Jogador` argumento à `Seleção` apenas se, ao assim fazer, não extrapolar o número máximo de membros. Analogamente, `dispensarJogador()` só dispensa o `Jogador` argumento se essa ação não reduzir o tamanho do elenco para abaixo do mínimo exigido. Em caso, de impossibilidade ambos os métodos lançam uma `IllegalStateException`.

Quanto a regra 3., ela é regida pela função `podeJogar()`, a qual retorna `true` se não houver jogadores `LESIONADOS` ou `SUSPENSOS` no `time`, retornando `false` caso contrário.

No tocante à persistência de dados, optou-se pela utilização de registros JSON, salvos em arquivos JSONL (JavaScript Object Notation in Line), em virtude da escalabilidade e fácil capacidade de inserção (`append`) fornecida por esse modelo de armazenamento. A implementação desta parte, se deu pela biblioteca Jackson, responsável pela captação e escrita de objetos `Json`, e por um pacote denominado `org.example.persistencia`, o qual agrupa as classes `IOJogador` e `IOSelecao`. Cada uma das classes tem rotinas de inserção, edição, exclusão e análise/leitura dos arquivos `.jsonl`, direcionadas para acessar, nessa ordem, os arquivos `jogadores.jsonl` e `selecoes.jsonl`.

Por fim, as exceções foram categorizadas em `IOException`, `IllegalArgumentException` e `IllegalStateException` e tratadas individualmente dentro de cada método que as gerou; visando fornecer mais detalhes sobre suas origem e tratamento.

2.3.3 Partidas, Fases e Resultados

Foram definidas regras de negócio específicas para o gerenciamento das partidas e das diferentes fases da competição:

1. Cada partida deve envolver exatamente duas seleções distintas.

2. Apenas seleções classificadas para determinada fase podem participar das partidas dessa fase.
3. Toda partida deve possuir um resultado registrado antes da geração dos classificados para a fase seguinte.
4. Na fase de grupos, a classificação das seleções deve respeitar o sistema de pontuação estabelecido pela competição.
5. Nas fases eliminatórias, cada partida deve obrigatoriamente produzir um vencedor.

A regra 1 é garantida durante a criação das partidas. Antes que uma nova partida seja registrada, o sistema verifica se as duas seleções informadas são diferentes. Caso o usuário tente cadastrar uma partida entre a mesma seleção, uma exceção do tipo `IllegalArgumentException` é lançada, impedindo o registro de dados inconsistentes.

A regra 2 é implementada através da organização das fases da competição. Cada fase possui um arquivo específico contendo as seleções classificadas. Dessa forma, ao criar partidas para uma determinada etapa do torneio, somente as seleções presentes no arquivo de classificados da fase correspondente podem ser utilizadas. Esse mecanismo garante a consistência do chaveamento ao longo da competição.

Quanto à regra 3, ela é garantida pelo método responsável pelo registro de resultados. As seleções somente podem ser promovidas à fase seguinte após todas as partidas da etapa atual possuírem placares devidamente cadastrados. Assim, evita-se a geração prematura de classificados e possíveis inconsistências nos confrontos futuros.

A regra 4 é aplicada exclusivamente na fase de grupos. Para cada partida concluída, o sistema atualiza a pontuação das seleções envolvidas segundo os critérios estabelecidos:

- Vitória: 3 pontos.
- Empate: 1 ponto para cada seleção.
- Derrota: 0 pontos.

Além da pontuação, são armazenadas estatísticas auxiliares, como gols marcados, gols sofridos e saldo de gols, utilizadas como critérios de desempate durante a classificação dos grupos.

Já a regra 5 é aplicada nas fases eliminatórias. Nessas etapas não é permitido que uma partida termine sem vencedor. Caso o placar termine empatado, o sistema exige a indicação da seleção vencedora nos pênaltis antes de concluir o registro do resultado. O vencedor é então automaticamente encaminhado para o arquivo de classificados da próxima fase.

No tocante à persistência de dados, optou-se pela utilização de arquivos JSON para armazenamento das informações referentes às partidas, resultados e seleções classificadas. Cada fase da competição possui arquivos próprios para registrar seus confrontos e seus classificados, facilitando a organização dos dados e a manutenção do sistema. A manipulação desses arquivos é realizada por meio da biblioteca Jackson, responsável pela serialização e desserialização dos objetos Java.

Do tratamento de exceções O módulo de partidas, fases e resultados utiliza tratamento de exceções para impedir a inserção de informações inválidas e preservar a integridade dos dados da competição.

Durante o cadastro de partidas, são verificadas situações como a seleção da mesma equipe para ambos os lados do confronto ou a tentativa de registrar seleções inexistentes na fase atual. Nesses casos, é lançada uma exceção do tipo `IllegalArgumentException` contendo uma mensagem explicativa ao usuário.

No registro de resultados, também são realizadas validações para impedir valores negativos de gols, partidas inexistentes ou tentativas de registrar resultados em confrontos que já possuam placar definido. Caso alguma dessas situações seja identificada, uma exceção apropriada é lançada e o registro não é realizado.

Adicionalmente, operações envolvendo leitura e escrita dos arquivos JSON são protegidas por blocos try-catch, permitindo o tratamento de possíveis falhas de entrada e saída (`IOException`). Dessa forma, erros relacionados à persistência dos dados são capturados adequadamente, evitando o encerramento inesperado da aplicação e possibilitando a exibição de mensagens informativas ao usuário.

2.3.4 Estádio e Arbitragem

Foram exigidas três regras de negócio específicas para o gerenciamento de estádios e árbitros da competição:

1. Um estádio não pode sediar duas partidas no mesmo horário.
2. Cada partida deve ter ao menos um árbitro principal.
3. Árbitros não podem atuar em partidas de seleções de sua nacionalidade.

A regra 1 é garantida durante o agendamento das partidas. O sistema realiza uma verificação de disponibilidade do objeto `Estadio` para a data e hora solicitadas. Caso o horário já esteja reservado para outro confronto, é lançada uma `IllegalArgumentException`, bloqueando o agendamento conflitante.

A regra 2 é validada no momento da instanciação da partida. O construtor exige a passagem de um objeto `Arbitro` válido. Se o atributo correspondente ao árbitro principal for nulo (`null`), o sistema interrompe a criação e alerta sobre a falta de equipe de arbitragem.

A regra 3 é assegurada pelo método `validarNacionalidadePartida()` implementado na classe `Arbitro`. Antes de vincular um árbitro a um jogo, o método compara o atributo `nacionalidade` do árbitro com a nacionalidade das duas seleções envolvidas. Em caso de equivalência, uma `IllegalArgumentException` é disparada para impedir conflitos de interesse.

No tocante à persistência de dados, optou-se pela utilização do formato textual JSON. Foi definido que os arquivos gerados fossem salvos no diretório padrão do projeto (`src/main/resources`). Para a serialização e desserialização, optou-se pela utilização da biblioteca externa Jackson. Através do padrão de projeto DAO, materializado em `GerenciadorEstadioJSON` e `GerenciadorArbitroJSON`, os dados inseridos pelos usuários são persistidos usando a classe `ObjectMapper`. Para a leitura, utilizou-se o `TypeReference` para contornar o apagamento de tipo e garantir a correta reconstrução das listas genéricas (`List<Estadio>`, `List<Arbitro>`).

Por fim, o tratamento de exceções atua como camada de robustez da interface gráfica.

- **NumberFormatException:** Adicionado nos campos de interface gráfica, como o de capacidade do estádio e o de experiência do árbitro, evitando a quebra do software ao receber strings. Caso ocorra essa exceção, é apresentada uma mensagem amigável via `JOptionPane`.
- **IOException:** Aplicado nas classes gerenciadoras a fim de contornar erros de leitura e gravação pelo sistema operacional. Ao capturar o erro, o sistema retorna listas vazias, permitindo que a aplicação inicie de forma estável mesmo sem um arquivo pré-existente.

2.3.5 Ingressos e Público

O módulo de Ingressos e Público foi desenvolvido com o objetivo de gerenciar a venda de ingressos para as partidas da competição, bem como controlar informações relacionadas ao público presente e à arrecadação financeira obtida em cada evento. Para isso, foram utilizadas as classes *Ingresso*, *DadosIngressos* e *OperacaoIngresso*, cada uma desempenhando uma responsabilidade específica dentro da aplicação.

Foram definidas as seguintes regras de negócio para este módulo:

1. Cada ingresso deve estar associado a uma partida específica.
2. A compra de ingressos somente pode ser realizada quando houver quantidade disponível.
3. A quantidade de ingressos disponíveis deve ser atualizada após cada venda realizada.
4. A arrecadação total deve ser calculada a partir da soma das arrecadações registradas para cada partida.

A classe *Ingresso* é a principal entidade do módulo. Ela é responsável por representar os ingressos disponíveis para uma determinada partida, armazenando informações como o nome do jogo, o preço unitário do ingresso e a quantidade disponível para venda. Dessa forma, cada objeto da classe representa um conjunto de ingressos relacionados a uma partida específica da competição.

A primeira regra de negócio é garantida pela própria estrutura da classe *Ingresso*, que mantém o atributo *jogo*. Esse atributo identifica a partida à qual o ingresso pertence, permitindo que o sistema organize e diferencie os ingressos de cada confronto cadastrado.

A segunda e a terceira regras são implementadas através do método *comprar()*. Quando uma solicitação de compra é realizada, o sistema verifica inicialmente se a quantidade solicitada pelo usuário é menor ou igual à quantidade de ingressos ainda disponível. Caso a condição seja satisfeita, a compra é concluída com sucesso e a quantidade disponível é reduzida automaticamente. Caso contrário, o sistema informa que não existem ingressos suficientes para atender à solicitação realizada.

A lógica de compra foi separada em uma classe específica denominada *OperacaoIngresso*. Essa classe atua como uma camada intermediária responsável por executar operações relacionadas aos ingressos. Atualmente, ela disponibiliza o método estático *comprarIngresso()*, que recebe um objeto da classe *Ingresso* e a quantidade desejada, delegando ao próprio ingresso

a execução da compra. Essa separação contribui para uma melhor organização do código e facilita futuras expansões do módulo.

Para o armazenamento das informações compartilhadas entre as diferentes telas do sistema, foi criada a classe *DadosIngressos*. Nela são mantidas as estruturas responsáveis pelo controle das partidas cadastradas, da quantidade de ingressos associada a cada partida e dos valores arrecadados ao longo da competição.

No desenvolvimento dessa classe, optou-se pela utilização das estruturas de dados *ArrayList* e *HashMap*. A lista de partidas é armazenada em um *ArrayList*, permitindo a manipulação dinâmica dos confrontos disponíveis no sistema. Já as informações referentes à quantidade de ingressos e aos valores arrecadados são armazenadas em *HashMaps*, utilizando o nome da partida como chave de acesso. Essa abordagem oferece consultas rápidas e eficientes, além de simplificar a atualização dos dados durante a execução da aplicação.

A quarta regra de negócio é implementada pelo método *getTotalGeral()*, responsável por percorrer todos os valores armazenados na estrutura de arrecadação e calcular a receita total obtida. Esse procedimento permite consolidar informações financeiras de diferentes partidas em um único resultado, facilitando a geração de relatórios e o acompanhamento do desempenho financeiro da competição.

Além das funcionalidades de processamento dos dados, o módulo também conta com uma interface gráfica desenvolvida utilizando a biblioteca Swing. A navegação entre as funcionalidades é centralizada pela classe *MenuPrincipal*, que disponibiliza acesso às telas de venda de ingressos, controle de público e relatórios financeiros. Essa organização permite que o usuário encontre facilmente as funcionalidades desejadas, tornando a utilização do sistema mais intuitiva.

Por fim, o módulo foi desenvolvido seguindo princípios da Programação Orientada a Objetos, com destaque para o encapsulamento dos atributos, a divisão de responsabilidades entre as classes e a utilização de estruturas de dados adequadas para armazenamento das informações. Essas decisões contribuíram para a organização do código, para a manutenção da aplicação e para a integração do módulo com as demais partes do sistema de gestão da Copa do Mundo.

2.4 Integração e navegabilidade

Texto sobre a entrega da Etapa 4.

3 Resultados e Discussão

O texto será adicionado apenas na entrega da versão final.

3.1 Tecnologia Utilizadas

O texto será adicionado apenas na entrega da versão final.

3.2 Demonstração da Solução

O texto será adicionado apenas na entrega da versão final.

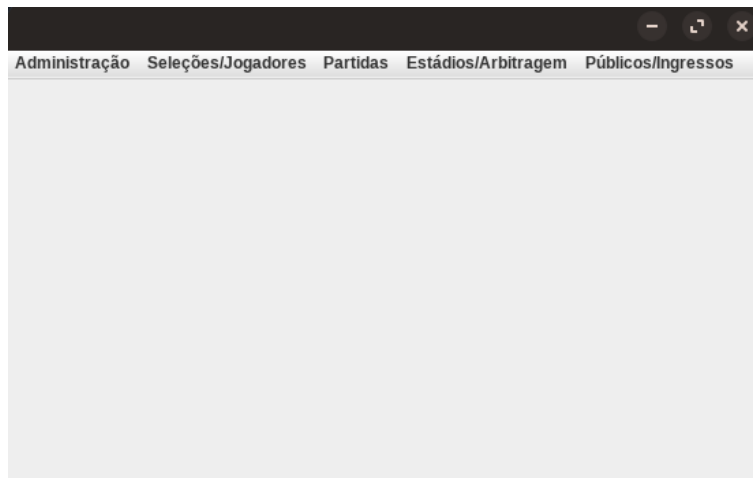
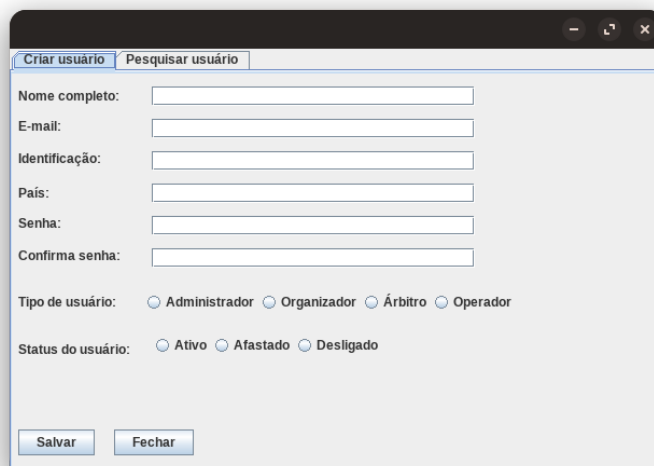


Figura 2: Tela Inicial

A screenshot of a login window titled "Login". It features two input fields: "E-mail:" and "Senha:". Below the "Senha:" field are two buttons: "Entrar" and "Esqueci minha senha". The window has a standard OS-style title bar with minimize, maximize, and close buttons.

Figura 3: Tela Login

3.2.1 Módulo Administrador

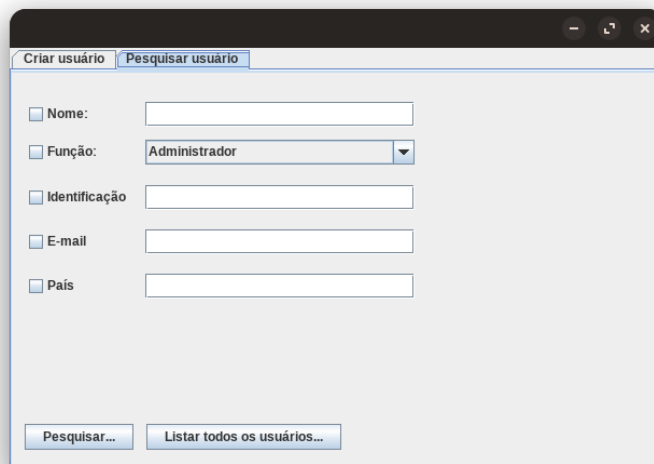


The 'Criar usuário' window features a tabbed interface with 'Criar usuário' and 'Pesquisar usuário'. The 'Criar usuário' tab contains the following fields and options:

- Nome completo:
- E-mail:
- Identificação:
- País:
- Senha:
- Confirma senha:
- Tipo de usuário: ☐ Administrador ☐ Organizador ☐ Árbitro ☐ Operador
- Status do usuário: ☐ Ativo ☐ Afastado ☐ Desligado

At the bottom are two buttons: 'Salvar' and 'Fechar'.

Figura 4: Tela de criação de usuário



The 'Pesquisar usuário' window features a tabbed interface with 'Criar usuário' and 'Pesquisar usuário'. The 'Pesquisar usuário' tab contains the following fields and options:

- ☐ Nome:
- ☐ Função:
- ☐ Identificação:
- ☐ E-mail:
- ☐ País:

At the bottom are two buttons: 'Pesquisar...' and 'Listar todos os usuários...'.

Figura 5: Tela de pesquisa de usuário

Usuários Seleções/Jogadores Partidas Estádios/Arbitragem Público/Ingressos

Número total de usuários: 0

Número de administradores: 0

Número de organizadores: 0

Número de operadores: 0

Número de árbitros: 0

Número de usuários ativos: 0

Número de usuários afastados: 0

Número de usuários desligados: 0

Baixar arquivo detalhado...

Figura 6: Tela de relatórios do administrador

3.2.2 Módulo Seleções/Jogadores

Cadastro de Jogador

Nome:

Posição:

Número:

Status:

Data de Nascimento (dd/mm/yyyy):

Salvar Cancelar

Figura 7: Tela de cadastro de jogador

3.2.3 Módulo Partidas

Cadastro de Partidas | Registro de Resultados | Consulta de Jogos

Partida 1:

Data da Partida :

Horário da Partida :

Nome do Estádio :

Seleção 1:

Seleção 2:

Árbitro :

Figura 10: Tela de criação de partida

Cadastro de Partidas | Registro de Resultados | Consulta de Jogos

Digite o Número da Partida:

Gols da Seleção 1:

Gols da Seleção 2:

(Caso haja um empate depois da Fase de Grupos) Qual seleção ganhou nos pênaltis? :

Figura 11: Tela de registro de partida

Cadastro de Partidas | Registro de Resultados | **Consulta de Jogos**

Tipo de Pesquisa: ☒ Por Número ☐ Por Seleções

Número da Partida* : Seleção 1*:

Seleção 2*:

Número da Partida:
 Seleção 1: Seleção 2:
 Gols Seleção 1: Gols Seleção 2:
 Vencedor:

Figura 12: Tela de consulta de partida

3.2.4 Módulo Estádios/Arbitragem

Cadastro Estádio

Nome:

Localização:

Capacidade:

Nome	Localização	Capacidade

Figura 13: Tela de cadastro de estádio

Cadastro Árbitro

Nome completo:

Nacionalidade:

Experiência:

Figura 14: Tela de cadastro de árbitro

Designação Árbitro

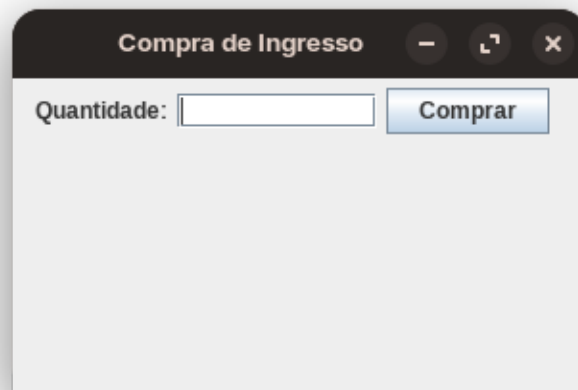
Árbitro:

Função:

Partida:

Figura 15: Tela de designação de árbitro

3.2.5 Módulo Público/Ingressos



Compra de Ingresso

Quantidade:

Comprar

Figura 16: Tela de compra de ingressos



Controle de Público	
Partida:	Brasil x Argentina
Capacidade:	50.000
Vendidos:	32.000
Público atual:	32.000

Figura 17: Tela de controle de público



Figura 18: Tela de relatórios financeiros

3.3 Técnicas de programação Aplicadas

O texto será adicionado apenas na entrega da versão final.

3.4 Desafios Encontrados e Soluções Adotadas

O texto será adicionado apenas na entrega da versão final.

3.5 Análise Crítica da Qualidade da Solução

O texto será adicionado apenas na entrega da versão final.

4 Conclusão e Evolução Futura

O texto será adicionado apenas na entrega da versão final.

Referências

[1]

[2]

[3]